

Differentiable Multidisciplinary Design Optimization of an Electric-Pump-Fed Liquid Rocket Engine

Formulation, Literature Basis, and Implementation Tracker for LREKit

Ibrahim Shahid

July 22, 2026 • Version 0.1 (living document)

Purpose. A single reference for the differentiable engine-MDO workstream: the mathematical formulation, the governing physics of each discipline, the literature each modelling choice rests on, and a phase/gate table to track build progress. It supersedes the loose plan notes and folds in the literature verification of July 2026.

Build. `latexmk-pdfDifferentiable_Engine_MDO_Plan.tex` (or `pdflatex` twice for the table of contents).

Status legend. ■ done complete ■ wip in progress ■ to do not started.

Contents

1	Scope and one-paragraph statement	2
2	Corrected literature foundation	2
2.1	The Rao 1958 vs. Rao 1999 correction (load-bearing)	2
2.2	Verified source identities	3
2.3	Verified discipline mappings	3
3	System boundary and mass ledger	4
4	Mathematical formulation	4
4.1	Variables and states	4
4.2	The MDF optimization problem	4
4.3	Total derivatives by implicit differentiation	4
4.4	Differentiating converged fixed points	5
5	Coupling structure	5
6	Disciplinary models and residual blocks	6
6.1	Nozzle and performance	6
6.2	Regenerative cooling	7
6.3	Injector and spray	7
6.4	Pumps and electrical system	8
7	The Rao existence boundary: a three-diagnostic framework	8
8	Objectives and constraints	9

9	Differentiability rules	9
10	Repository architecture	10
11	Implementation phases and acceptance gates	10
12	Implementation and sequencing	11
12.1	Four rules that keep it tractable	11
12.2	Phase-to-code mapping	12
12.3	The coupled-solve core (Phase 7)	12
12.4	Two parallel tracks	13
12.5	First increment (the walking skeleton)	13
13	Open risks and decision log	13

1 Scope and one-paragraph statement

We build a *multidisciplinary-feasible* (MDF), end-to-end differentiable model of the continuous states of an electric-pump-fed liquid rocket engine, and drive it with a gradient-based constrained optimizer. Discrete hardware choices (channel/blade/stage counts, materials, architecture) are handled by an *outer enumeration*, not by relaxing integers. “End-to-end differentiable” means *exact total derivatives of the discretized engineering models*; it does *not* mean differentiating CAD Boolean operations, integer counts, or arbitrary software branches. The engineering novelty is the coupled *total* derivative through the converged engine state; the physical novelty is the interaction of engine-level optima with the Rao nozzle *existence boundary* under perfect- and real-gas thermochemistry.

2 Corrected literature foundation

The plan’s architecture is justified paper-by-paper. The mappings below were checked against the source PDFs in `propulsion_texts/` (text extracted and read, not inferred from filenames). One load-bearing attribution was corrected.

2.1 The Rao 1958 vs. Rao 1999 correction (load-bearing)

Correction. The claim “the contour must remain an implicit solved state, not a Bézier lookup” is *physically correct* but was attributed to the wrong source. **Rao 1958** [1] establishes that the optimum contour is a constrained variational solution — the control surface is forced to be a characteristic surface and thrust is maximized at fixed mass flow via Lagrange multipliers λ_2, λ_3 , with Rao stating the optimality condition “is *implicit* in the solution” of the governing equations. But Rao 1958 does *not* require an engine MDO to re-solve the contour implicitly at every iterate; the Rao/TOP parabola is a *validated approximation* to that same optimum. The requirement for the implicit contour comes from **Rao 1999** [2], which defines the *boundary of the valid (shock-free) region* — the “invalid region” where the optimum-thrust control surface can no longer be computed normally. *That boundary is the “Rao cliff”*; a Bézier lookup has no such boundary. Two consequences: (i) cite Rao 1999, not Rao 1958, for the implicit-contour requirement; (ii) Rao 1999 already computes how *equilibrium chemistry shifts the valid-region boundary*, so it is direct prior art for RQ4 (real-gas migration).

Design consequence — two contour fidelities, switched by the question.

- *Bulk MDO sweeps* (mass- I_{sp} Pareto front): use the analytic Rao/TOP Bézier contour. It is smooth, cheap, endpoint-exact, and adequate for the value-of-coupling questions.
- *Existence-boundary and exact-vs-TOP studies*: use the implicit variational/MOC boundary-value contour as a solved state. Total derivatives to the *manufacturable wall* here still require the differentiable kernel/BDE march (the “J3b” item); until that lands, the kernel *BD* is a frozen artifact and wall-coordinate sensitivities are unavailable. Keep this on the critical path only for the runs that need it.

2.2 Verified source identities

Cited as	File	Verified identity
NASA SP-8087	19730022965.pdf	“Liquid Rocket Engine Fluid-Cooled Combustion Chambers” — confirmed (text layer).
NASA SP-8109	fuel_pump_design/19740020848.pdf	“...Centrifugal Flow Turbopumps” — confirmed.
NASA SP-8052	fuel_pump_design/19710025474.pdf	“...Turbopump Inducers” — confirmed.
NASA SP-8089	pintle_injector/19760023196.pdf	<i>Image scan, no text layer</i> — identity not text-verifiable; consistent with “...Injectors”. Requires OCR to parse.
Rao 1958	rao1958.pdf	“Exhaust Nozzle Contour for Optimum Thrust”. Page 1 is a spillover references page from the preceding journal article; Rao’s text begins p. 2.
Rao 1999	rao1999.pdf	“Nozzle Optimization for Space-Based Vehicles” — valid/invalid region, equilibrium-chemistry boundary.

2.3 Verified discipline mappings

Source	Claim used in plan	Verification
Pizzarelli 2011 [14]	Channel aspect ratio governs stratification, rib conduction, wall temperature, Δp ; throat-only thermal is insufficient.	Confirmed: quasi-2-D (1-D coolant mass/momentum + 2-D energy), $k = h/b$ up to 8 at throat; higher k lowers hot-gas wall temperature via ribs.
Son 2017 [16]	Real minimum-area transition (tip-opening vs. center-gap limited); model as two branches, not <code>min()</code> .	Confirmed: center-gap area vs. minimum orifice near pintle tip; explicit “transition of the minimum area”.
Lee 2021 [18]	Battery mass has separate power- and energy-limited terms; burn duration is an optimization variable.	Confirmed: power density and energy/discharge-time both drive battery mass; voltage droop at short discharge.
SP-8087 [6]	Regen cooling as coupled allocation of coolant, Δp , wall temperature, geometry, life.	Confirmed.
SP-8109/8052 [8, 9]	Pump RPM and inducer suction/mechanics as a simultaneous constrained design.	Confirmed.

3 System boundary and mass ledger

The optimized system runs from the tank outlets and electrical source to the nozzle exit. Everything excluded must appear explicitly (as zero or as a fixed allowance) in the ledger, so tanks and trajectory can be added later without changing the engine formulation.

Inside boundary	Initially excluded (ledger placeholders)
Fuel/oxidizer pumps, inducers, motors, inverters, battery	Propellant tanks, pressurant
Feed-line and manifold pressure losses	Vehicle structure, TVC
Regenerative cooling circuit	Valves, ignition, avionics
Pintle injector, chamber, converging section	Transient combustion CFD
Rao nozzle and delivered performance	Blade-resolved pump CFD, CAD Booleans

Reported masses: thrust-chamber/nozzle hardware, injector, pumps+inducers, motors+inverters, battery, total installed propulsion-package mass, and mission propellant consumed.

4 Mathematical formulation

4.1 Variables and states

Let a be discrete architecture choices, x_h shared continuous hardware variables, x_k controls at operating point k , and y_k the coupled physical states at point k :

$$x_h = [P_c, O/F, \varepsilon, L\%, R_d/R_t, L^*, A_c/A_t, t_{\text{hot}}, w_c, h_c, \beta_{\text{regen}}, \frac{\Delta p_f}{P_c}, \frac{\Delta p_o}{P_c}, D_{\text{pintle}}, N_f, N_o], \quad (1)$$

where N_f, N_o are continuous pump speeds (not blade counts). The minimum implicit state is

$$y = [u_{\text{Rao}}, R_t, \eta_{c^*}, T_{c,i}, P_{c,i}, q_i, T_{wg,i}, T_{wc,i}, Q_f, H_f, Q_o, H_o]. \quad (2)$$

Relations that are naturally explicit stay explicit; we do not enlarge the nonlinear system merely to appear “coupled.”

4.2 The MDF optimization problem

$$\begin{aligned}
 & \min_{a, x_h, \{x_k\}} J(x_h, \{x_k\}, \{y_k\}) \\
 & \text{s.t.} \quad R_k(y_k, x_h, x_k; a) = 0 \quad (\text{engine state, each point } k) \\
 & \quad \quad g_k(y_k, x_h, x_k; a) \leq 0 \quad (\text{local constraints}) \\
 & \quad \quad G_{\text{mission}}(\{y_k\}, x_h) \leq 0. \quad (\text{shared/mission constraints})
 \end{aligned} \quad (3)$$

MDF: every optimizer evaluation receives a *converged* engine state $y_k(x)$ satisfying $R_k = 0$. This is the standard multidisciplinary-feasible architecture [22, 27].

4.3 Total derivatives by implicit differentiation

We never differentiate through solver iterations. At a converged state $R(y, x) = 0$ the implicit function theorem gives

$$R_y \frac{dy}{dx} + R_x = 0 \implies \frac{dy}{dx} = -R_y^{-1} R_x, \quad (4)$$

so for any output $f(y, x)$,

$$\frac{df}{dx} = f_x - f_y R_y^{-1} R_x. \quad (5)$$

The bracketed inverse is realized in one of two modes. **Direct/forward** solves $R_y \phi = -R_x$ for the state sensitivities $\phi = dy/dx$ (cost $\propto n_x$ linear solves). **Adjoint/reverse** solves

$$R_y^\top \lambda = f_y^\top, \quad \frac{df}{dx} = f_x - \lambda^\top R_x, \quad (6)$$

at cost $\propto n_f$ solves (one per objective/aggregated constraint). This is the unified-derivatives view of Martins & Hwang and the MAUD architecture in OpenMDAO [23, 24, 25].

Mode selection (honest sizing). The first engine has $n_x \approx 12$ –20 continuous variables but *hundreds* of stationwise constraints. This is **not** high-dimensional MDO: adjoint has no asymptotic edge here. Use *forward/direct* total derivatives to assemble the full constraint Jacobian in one pass; use the *adjoint* only for scalar objectives. The real win over finite differences is exactness and freedom from step-size noise, plus tight KKT satisfaction — not conquering dimensionality.

4.4 Differentiating converged fixed points

Feedback closures (the η_{c^*} loop, the pump operating point) are fixed points $y = T(y, x)$ or roots $F(y, x) = 0$. Differentiate the *converged* root via (4), implemented with a `custom_root/ custom_fixed_point` construct [28, 29], never by unrolling the iteration. Inner residuals must be converged at least $100\times$ tighter than the optimizer feasibility tolerance, or the “exact” gradient is exact for the wrong equation.

5 Coupling structure

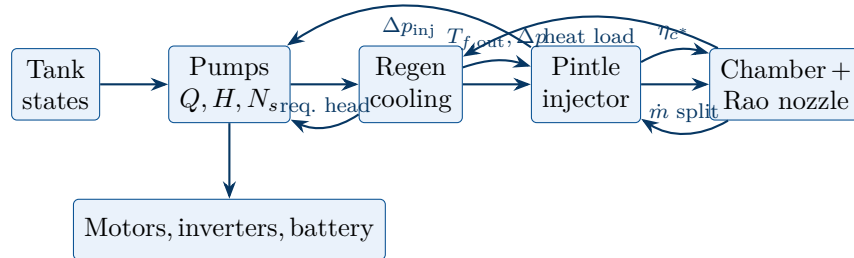


Figure 1: Engine coupling graph. The chain is left-to-right; feedback edges close the loops that make one-pass sequential design suboptimal.

The two dominant loops are the *combustion/mass-flow* loop

$$\eta_{c^*} \rightarrow \dot{m} \rightarrow \text{injector/spray} \rightarrow \eta_{c^*}, \quad (7)$$

and the *hydraulic/electrical* loop

$$(\Delta p_{\text{regen}} + \Delta p_{\text{injector}} + P_c) \rightarrow H_{\text{pump}} \rightarrow (P_{\text{electric}}, m_{\text{pump}}, m_{\text{battery}}). \quad (8)$$

Strongest edge, weakest physics. The η_{c^*} loop dominates RQ1’s “value of coupling” *and* is the least trustworthy link — in the current repository it is a one-way correlation fixed point, not energy-closed. Run an ablation with η_{c^*} held fixed to bound how much of any Pareto gain rests on that correlation, and report the spray- c^* coupling value as bounded below by correlation fidelity.

6 Disciplinary models and residual blocks

Throughout, gas is calorically/thermally described by a fixed-composition property model; equilibrium chemistry enters only through precomputed property surfaces (§9).

6.1 Nozzle and performance

Ideal quasi-1-D core. With ratio of specific heats γ , the isentropic area–Mach relation and pressure ratio are

$$\frac{A}{A^*} = \frac{1}{M} \left[\frac{2}{\gamma+1} \left(1 + \frac{\gamma-1}{2} M^2 \right) \right]^{\frac{\gamma+1}{2(\gamma-1)}}, \quad \frac{p}{p_c} = \left(1 + \frac{\gamma-1}{2} M^2 \right)^{-\frac{\gamma}{\gamma-1}}. \quad (9)$$

Define the Vandekerckhove function $\Gamma(\gamma) = \sqrt{\gamma} \left(\frac{2}{\gamma+1} \right)^{\frac{\gamma+1}{2(\gamma-1)}}$. Characteristic velocity and the mass-flow closure are

$$c_{\text{ideal}}^* = \frac{\sqrt{RT_c}}{\Gamma(\gamma)}, \quad \boxed{\dot{m} = \frac{P_c A_t}{\eta_{c^*} c_{\text{ideal}}^*} = \frac{P_c A_t}{c_{\text{delivered}}^*}} \quad (10)$$

with the convention pinned as $c_{\text{delivered}}^* = \eta_{c^*} c_{\text{ideal}}^*$ so the CEA property surface and the injector closure never double-count η_{c^*} . The thrust coefficient and target-thrust closure are

$$C_F = \underbrace{\sqrt{\frac{2\gamma^2}{\gamma-1} \left(\frac{2}{\gamma+1} \right)^{\frac{\gamma+1}{\gamma-1}} \left[1 - \left(\frac{p_e}{p_c} \right)^{\frac{\gamma-1}{\gamma}} \right]}}_{\text{momentum}} + \underbrace{\frac{p_e - p_a}{p_c}}_{\text{pressure}} \varepsilon, \quad F_{\text{target}} - C_F P_c \pi R_t^2 = 0, \quad (11)$$

and $I_{sp} = C_F c_{\text{delivered}}^* / g_0$ (see [4, 12, 5]).

Rao optimum contour (implicit fidelity). The maximum-thrust wall is the variational solution of Rao [1]: along the control surface DE the flow is isentropic and the surface coincides with a characteristic; stationarity plus mass continuity and the axisymmetric compatibility relation

$$\frac{dr}{dx} = \tan(\theta \pm \mu), \quad d(\theta \mp \nu) = \mp \frac{\sin \theta \sin \mu}{r} ds \quad (\mu = \arcsin \frac{1}{M}, \nu = \text{Prandtl-Meyer}) \quad (12)$$

close the B – D – E topology, with Lagrange multipliers enforcing the mass/length constraints. The residual block is

$$R_{\text{Rao}}(u_{\text{Rao}}; \varepsilon, L\%, \gamma, R_d/R_t) = 0. \quad (13)$$

Valid region / the “cliff” (Rao 1999). A shock-free optimum control surface exists only inside the *valid region*; on its boundary the right characteristic through D reaches a caustic/envelope and the construction fails [2]. This boundary — *not* a Bézier artifact — is the object of RQ3/RQ4 and is tracked by the three diagnostics of §7.

6.2 Regenerative cooling

Gas-side heat transfer. The Bartz correlation [11] gives the hot-gas film coefficient

$$h_g = \left[\frac{0.026}{D_t^{0.2}} \right] \left[\frac{\mu^{0.2} c_p}{Pr^{0.6}} \right] \left[\frac{p_c g_0}{c^*} \right]^{0.8} \left(\frac{D_t}{R_c} \right)^{0.1} \left(\frac{A_t}{A} \right)^{0.9} \sigma, \quad (14)$$

with σ the property-variation correction. The driving temperature is the recovery (adiabatic-wall) temperature

$$T_{aw} = T_{c,\text{gas}} \frac{1 + r \frac{\gamma-1}{2} M^2}{1 + \frac{\gamma-1}{2} M^2}, \quad r = Pr^{1/3} \text{ (turbulent recovery factor)}. \quad (15)$$

Series thermal circuit and stationwise closures. Per station i along a fixed-topology normalized axial grid (so JAX array shapes never change),

$$q = \frac{T_{aw} - T_c}{1/h_g + R_{\text{wall}} + 1/h_c}, \quad \dot{m}_c c_{p,c} \frac{dT_c}{ds} = q P_{\text{heated}}, \quad (16)$$

$$\frac{dP_c}{ds} = - \underbrace{f \frac{1}{D_h} \frac{\rho u^2}{2}}_{\text{friction}} - \underbrace{\rho u \frac{du}{ds}}_{\text{acceleration}} - \Delta p_{\text{minor}}, \quad (17)$$

with hydraulic diameter D_h and Darcy factor f from a Colebrook/Churchill fit [13]. **Aspect-ratio (HARCC) effect** enters the wall resistance through fin efficiency of the ribs,

$$\eta_{\text{fin}} = \frac{\tanh(m L_{\text{fin}})}{m L_{\text{fin}}}, \quad m = \sqrt{\frac{2h_c}{k_w t_{\text{fin}}}}, \quad (18)$$

which is precisely the first-order mechanism Pizzarelli's quasi-2-D model resolves in full [14, 15].

Thermal fidelity ladder (do not gate on the hard version first). *4a* — 1-D series resistance + fin efficiency (captures the leading aspect-ratio trend; already in the repo as Bartz + fin correction). *4b* — Pizzarelli-style quasi-2-D wall/coolant energy as the *final* discipline. A differentiable quasi-2-D cross-section solve at every station is the single most expensive discipline; stage it, and let *4a* carry first coupled solves.

6.3 Injector and spray

Each stream obeys the orifice equation

$$\dot{m}_j = C_{d,j} A_j \sqrt{2\rho_j \Delta p_j}, \quad (19)$$

areas being dependent geometry while $\Delta p_j/P_c$ and D_{pintle} are design variables. The total momentum ratio and blockage factor are

$$\text{TMR} = \frac{\dot{m}_r U_r}{\dot{m}_a U_a}, \quad \text{BF} = \frac{N w}{\pi D_p}, \quad \theta_{\text{spray}} \approx \arctan(\text{TMR}) \text{ (leading order)}, \quad (20)$$

matching Son/Hwang [16, 17]. The **movable-pintle minimum area** switches physically between tip-opening-limited (A_{mo}) and center-gap-limited (A_{cg}); this is modelled as *two smooth subproblems*,

each carrying a consistency inequality $A_{\text{active}} \leq A_{\text{other}}$, and the valid branches are compared afterward — *not* a differentiable $\min()$ [16]. A combustion-stability stand-in (the correctly deferred transient problem) is the stiffness screen

$$\Delta p_{\text{inj}}/P_c \geq 0.2, \quad (21)$$

a classical chug-avoidance heuristic [7, 10].

6.4 Pumps and electrical system

The per-propellant discharge requirement is the pressure ledger

$$\Delta p_{\text{pump}} = P_c + \Delta p_{\text{injector}} + \Delta p_{\text{regen}} + \Delta p_{\text{line}} - P_{\text{tank}} \quad (22)$$

(the oxidizer line omits Δp_{regen} unless explicitly cooled). Meanline performance uses the Euler head, specific and suction specific speeds, and NPSH [8, 9]:

$$H = \frac{U_2 c_{\theta 2} - U_1 c_{\theta 1}}{g_0}, \quad N_s = \frac{N \sqrt{Q}}{H^{3/4}}, \quad N_{ss} = \frac{N \sqrt{Q}}{\text{NPSH}^{3/4}}, \quad \text{NPSH} = \frac{p_{\text{in}} - p_v}{\rho g_0} + \frac{V^2}{2g_0}. \quad (23)$$

Electrical power and the required pump speed trade impeller size/mass, hydraulic efficiency, tip speed/rotor stress, NPSH, and motor torque/current:

$$P_{\text{electric}} = \frac{\dot{m} \Delta p_{\text{pump}}}{\rho \eta_{\text{pump}} \eta_{\text{motor}} \eta_{\text{inv}}}. \quad (24)$$

Efficiency model must be C^1 . The current piecewise/binning efficiency estimator cannot live inside the differentiable core; replace it with the smooth meanline-loss model or a C^1 surface over (N_s, D_s, ϕ) . Guard the pump operating point against branch-hopping, or the derivative is discontinuous.

Battery as an epigraph (Lee 2021 [18]): with specific energy ρ_E and specific power ρ_P ,

$$m_b \geq \frac{\sum_k P_{e,k} \Delta t_k}{\eta_{\text{discharge}} \rho_E}, \quad m_b \geq \frac{P_{e,k}}{\rho_P} \quad \forall k. \quad (25)$$

The objective selects whichever bound governs, with no differentiation through a hard $\max()$. Because the Pareto shape is sensitive to ρ_E , sweep it as a scenario band.

7 The Rao existence boundary: a three-diagnostic framework

Three *distinct* failures must not be conflated; they can trigger at different points in $(\varepsilon, L\%, \gamma)$:

1. **Numerical solvability** — smallest singular value $\sigma_{\min}(R_u)$ of the Rao residual Jacobian $\rightarrow 0$ (branch fold / loss of local solvability).
2. **Physical validity** — the Rao 1999 valid-region boundary: a characteristic caustic/envelope condition on the flow field, *independent* of the discretization [2].
3. **Optimality** — smallest eigenvalue $\lambda_{\min}(H_{\text{red}})$ of the reduced Hessian of the Rao Lagrangian $\rightarrow 0$ (loss of local optimality).

Continuation. Map the smooth solution through folds by pseudo-arclength continuation [30]: augment $R(u, p) = 0$ (parameter p) with

$$t_k^\top \left(\begin{bmatrix} u \\ p \end{bmatrix} - \begin{bmatrix} u_k \\ p_k \end{bmatrix} \right) - \Delta s = 0, \quad (26)$$

using a tangent predictor and Newton corrector so tracking survives where parameter stepping fails.

Soft mode. At the optimality boundary compute the eigenvector of $\lambda_{\min}(H_{\text{red}})$ and map it onto the contour to locate where the wall first becomes weakly determined.

Mesh-convergence gate. H_{red} is built on n_{control} nodes; its lowest mode can be a discretization artifact (checkerboard). Require $\lambda_{\min}$ and the eigenvector *shape* to converge under node doubling before any physical interpretation.

Optimizer discipline. A branch failure is a scientific result: the optimizer must *never* silently replace a failed Rao solution with a Bézier contour — mark the candidate infeasible.

8 Objectives and constraints

Use the ϵ -constraint method rather than a weighted sum:

$$\min m_{\text{package}} \quad \text{s.t.} \quad I_{sp, \text{mission}} \geq I_{sp, \text{min}}, \quad (27)$$

sweeping $I_{sp, \text{min}}$ to trace the mass–performance frontier. Constraint families: thrust equality at every commanded point; nozzle separation at every ambient pressure; stationwise gas- and coolant-side wall temperature; coolant phase/coking and Δp closure; liner stress, buckling, and a low-cycle fatigue screen (Coffin–Manson) [19, 20]; injector feature sizes, TMR, spray-wall clearance; pump NPSH, tip speed, stress, stable branch; motor torque/current/thermal; battery current/power/energy (25).

Replace max-type constraints by stationwise inequalities or a verified Kreisselmeier–Steinhauser aggregate

$$\text{KS}_\rho(g) = \frac{1}{\rho} \ln \sum_i e^{\rho g_i} \geq \max_i g_i. \quad (28)$$

Final optimization uses hard nonlinear constraints; penalty objectives are admitted only for initialization/feasibility restoration.

9 Differentiability rules

Difficulty	MDO treatment
Channel/slot/stage/blade counts	Outer enumeration (never relaxed).
max() wall temperature	Stationwise inequalities or verified KS aggregate.
Battery power/energy max()	Epigraph variable + inequalities (25).
Property phase switch	Stay in one phase via an explicit margin constraint.
Pintle minimum-area transition	Two active-branch subproblems, compared after.
Pump efficiency bins	Smooth meanline model or C^1 surface.
CEA/CoolProp calls	Precomputed C^1 shape-preserving property surfaces over $(P_c, O/F)$ and (T, P) ; re-evaluate final designs on the authoritative backends [21].
Rao→Bézier fallback	Mark infeasible; never silently substitute.
clip() hiding invalid physics	Physical bounds or regime constraints.
CAD/manufacturing Booleans	Post-optimum generation only.

10 Repository architecture

The reporting/CAD workflow (`design_nozzle_v2`) is untouched. The MDO gets a separate pure-numerical layer; CAD stays downstream of optimization.

```
raosim/mdo/
  schema.py      mission, architecture, variables, bounds
  scaling.py     variable / residual / constraint scaling
  properties.py  differentiable thermo + fluid surfaces (C1)
  grid.py        fixed-topology chamber/nozzle station grid
  nozzle.py      Rao adapter + performance (TOP-Bezier | implicit)
  cooling.py      thermal-hydraulic residuals (4a: 1-D+fin, 4b: quasi-2-D)
  injector.py    pintle geometry, flow, spray constraints (branch-split)
  pump.py        pump / inducer / motor / battery model
  mass.py        complete mass ledger
  assembly.py    full engine residual vector R(y,x)
  solve.py       coupled nonlinear state solve (block-sparse direct)
  derivatives.py forward, adjoint, Hessian-vector products (IFT, no unroll)
  nlp.py         SciPy trust-constr / SLSQP / IPOPT adapters
  multipoint.py  shared hardware + per-point controls
  postprocess.py -> LREKit result / CAD conversion
```

For the few-hundred-state system, assemble the block-sparse Jacobian and use a *direct* factorization: monolithic direct solves are typically fastest for systems of many inexpensive components [25, 26]. The Optimistix implicit-solve machinery already in `raosim/jax/bvp.py` is the correct starting point [29].

11 Implementation phases and acceptance gates

Numerical “done” means: implicit states solved $\geq 100\times$ tighter than optimizer feasibility; total directional derivatives agree with re-solved central differences to $\sim 10^{-4}$ in smooth regions; normalized constraint violation $< 10^{-5}$; KKT residual $\lesssim 10^{-4}$; doubling cooling/nozzle stations changes objective and active margins by $< 0.5\%$; several initial designs converge to the same solution or expose documented basins.

#	Deliverable	Completion gate	Status
0	Canonical 13 kN LOX/RP-1 architecture + complete mass boundary	Sequential LREKit design closes thrust, flow, feed pressure, mass ledger	■ done
1	Pure JAX data schema + explicit algebra	<code>jit</code> , <code>jacfwd</code> , <code>jacrev</code> work with no host callbacks	■ to do
2	Differentiable CEA/CoolProp property surfaces	Values and gradients pass held-out backend comparison	■ to do
3	Rao/performance total derivatives	AD matches FD of a fully re-solved Rao contour (<i>implicit-fidelity path; needs J3b for wall coords</i>)	■ wip
4a	1-D+fin regen residual	Energy, Δp , wall-temp, structural closures converge stationwise	■ to do
4b	Quasi-2-D regen (final discipline)	Reproduces Pizzarelli HARCC wall-temp/ Δp trends within tolerance	■ to do
5	Differentiable pintle/feed model	Areas, TMR, geometry, pressure ledger match NumPy within declared tol; branches split	■ to do

#	Deliverable	Completion gate	Status
6	Differentiable pump/electric model	Head, power, NPSH, geometry, mass match detailed model; C^1 efficiency	■ to do
7	Coupled engine solve	All residual blocks converge without manual sequential iteration	■ to do
8	Total derivative API	Directional derivatives of the resolved engine pass step-size sweeps	■ to do
9	Hard-constrained NLP	Scaled feasibility + KKT pass; no penalty-only “solution”	■ to do
10	Multipoint mission model	Shared hardware meets ambient, throttle, power, energy constraints at all points	■ to do
11	Authoritative re-evaluation	CEA/CoolProp/high-fidelity screens confirm the optimum or trigger a correction iteration	■ to do

Optimizer exploits model error (Phase 11). Gradient optimization against low-order screens drives designs into regions where those screens are least accurate. Expect some Pareto degradation at higher fidelity; mitigate by constraining designs to each screen’s calibration envelope, adding discrepancy margins, or a trust-region/multifidelity correction. Frame a partial-degradation result as a finding, not a failure.

12 Implementation and sequencing

The phase table (§11) is the *definition of done*; this section is the *build order*. Much of the differentiable machinery already exists: `raosim/jax/primitives.py` holds the analytic gas dynamics, `raosim/jax/thermal.py` holds Bartz/Sieder–Tate/Schmucker and a throat wall-temperature solve, and `raosim/jax/bvp.py` (line 69, `make_differentiable_solution`) is already an implicit-function-theorem wrapper. Implementation is therefore mostly *porting existing NumPy models into differentiable residual blocks and coupling them*, not writing physics from scratch.

Build principle — walking skeleton before depth. The dominant risk in a project this size is *integration*, not any single discipline. Build a thin end-to-end coupled solve first, with a low-fidelity stub for every discipline, so gradients flow from x to J and one Pareto point emerges; only then deepen each block (quasi-2-D cooling, implicit contour, real-gas surfaces). Consequently the recommended build order interleaves depth *after* the skeleton closes, and does not follow the phase numbering linearly.

12.1 Four rules that keep it tractable

1. **Wrap, don’t rewrite.** Every differentiable block is a `jax.numpy` port of an existing, audited NumPy model, kept beside its original as a *parity oracle* — the established `outputs_M3.5Perf/march-parity` pattern. You are re-expressing physics and proving $\sim 10^{-10}$ parity, not re-deriving it.
2. **One residual signature.** Everything conforms to $R(y, x) = 0$ and stacks into a single vector in `mdo/assembly.py`, exactly as `raosim/jax/assembly.py` already assembles the Rao residual.

3. **IFT at the seam.** The coupled solve is wrapped in `make_differentiable_solution` so the converged state is differentiable without unrolling the Newton iteration (§4, Eq. (4)).
4. **Parity + finite-difference gate on every block.** Each phase’s acceptance test is two comparisons: NumPy-vs-JAX parity, and AD-vs-central- difference agreement to $\sim 10^{-4}$. This is the gate column of §11, made concrete.

12.2 Phase-to-code mapping

Each block wraps named, existing code. New files live in the pure-numerical `raosim/mdo/` layer; the reporting/CAD path (`design_nozzle_v2`) is untouched and becomes the one-pass baseline for WP6.

#	Wraps / builds on (exists today)	New in <code>raosim/mdo/</code> + differentiability
1	<code>design.py</code> dataclasses (<code>DesignInput</code> , spec types)	<code>schema.py, scaling.py</code> : variable/bounds/mission pytrees; <code>jit/jacfwd</code> smoke test with no host callbacks.
2	<code>cea.py</code> (<code>cea_propellant</code> , <code>resolve_thermochemistry</code>), <code>coolants.py</code> , <code>frozen_flow.py</code>	<code>properties.py</code> : sample offline $\rightarrow C^1$ monotone splines over $(P_e, O/F)$ and (T, p) ; constrain optimizer to the tabulated box.
3	<i>already</i> JAX: <code>jax/primitives.py</code> (<code>area_mach_relation</code> , <code>thrust_coefficient</code>), <code>jax/sensitivities.py</code> (<code>cf_de_jax</code>)	<code>nozzle.py</code> : analytic TOP path is done; implicit path calls <code>solve_rao_bvp_jax</code> (host-only solve).
4a	<code>jax/thermal.py</code> (<code>bartz_hg</code> , <code>recovery_temperature</code> , <code>sieder_tate_hc</code> , <code>throat_wall_temperature</code>); oracle <code>thermal_design.py</code> : <code>size_cooling_channels</code>	<code>cooling.py</code> : march wall temperature on a fixed-topology station grid; fin efficiency carries the aspect-ratio trend.
4b	Pizzarelli quasi-2-D structure; <code>thermal_design.py</code> oracle	<code>cooling.py</code> (deepen): 2-D wall/coolant energy per station — the expensive swap, <i>after</i> the skeleton.
5	<code>injector.py</code> (<code>couple_atomization_to_performance</code> , <code>FeedSystemLedger</code> , <code>StabilityScreen</code> , <code>ManifoldDistribution</code>)	<code>injector.py</code> : orifice/TMR algebra is JAX-trivial; movable-pintle branch-split as two subproblems with consistency inequalities.
6	<code>pumps.py</code> meanline + <code>size_electric_pumps</code> ; <i>replace</i> the binned <code>estimate_pump_efficiency</code>	<code>pump.py</code> : C^1 meanline-loss efficiency surface; battery power/energy epigraph variables, Eq. (25).
7	<code>jax/bvp.py</code> (<code>least_squares_solve</code> , <code>make_differentiable_solution</code>)	<code>assembly.py, solve.py</code> : monolithic Newton, block-sparse direct factorization, IFT wrap.
8	<code>jax/api.py</code> , <code>jax/sensitivities.py</code>	<code>derivatives.py</code> : forward/direct for the full constraint Jacobian; adjoint for the scalar objective.
9	<code>jax/design_opt.py</code> (<code>constrained_nozzle_design</code> seed)	<code>nlp.py</code> : hand exact Jacobians to SciPy <code>trust-constr/SLSQP</code> , then IPOPT.
10	<code>separation.py</code> , <code>altitude_performance.py</code> , <code>trajectory.py</code>	<code>multiptoint.py</code> : shared x_h , per-point x_k, y_k ; separation and η surfaces evaluated at <i>every</i> operating point.
11	<code>cea.py</code> , <code>frozen_flow.py</code> , existing screens	<code>postprocess.py</code> : re-run optima on the authoritative backends; discrepancy-correction loop.

12.3 The coupled-solve core (Phase 7)

`assembly.py` stacks the per-discipline residuals into one $R(y, x)$; `solve.py` runs a damped Newton whose linear step is a *block-sparse direct factorization* — fastest for a few-hundred-state system of inexpensive components [25, 26]. Two needs your own notes already predict: (i) the feedback cycles of §5 make the Jacobian stiff, so budget a homotopy/continuation to *converge the state* at each optimizer iterate, independent of the cliff study; (ii) the η_{c^*} and pump operating-point fixed points must be inner `custom_root` solves converged 100× tighter than the outer feasibility tolerance, or the Phase-8 FD gate fails for the wrong reason.

12.4 Two parallel tracks

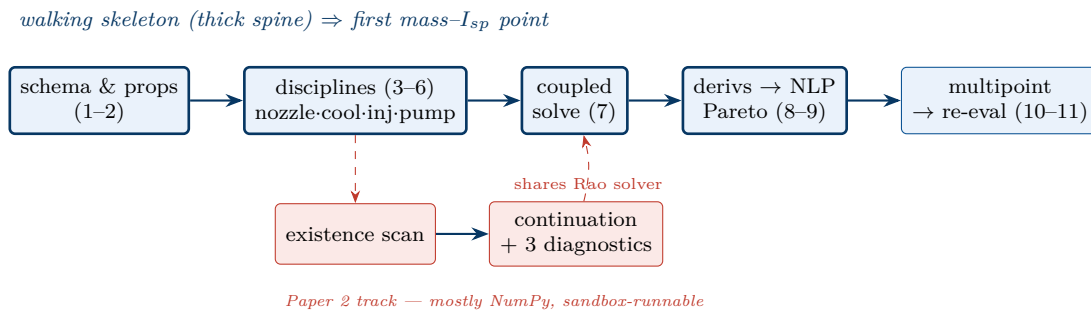


Figure 2: Build-dependency graph. The thick spine is the walking skeleton to a first mass- I_{sp} point; the red track (existence boundary) builds on `rao_existence_scan.py` and runs largely in parallel.

MDO Pareto (Paper 1) is the spine above: the mass- I_{sp} frontier is generated by ϵ -constraint sweeps in `nlp.py`, benchmarked against `design_nozzle_v2` (one-pass) and a block-coordinate loop over the *same* models (the fair baseline). **Rao cliff (Paper 2)** builds on `rao_existence_scan.py` — which already encodes the smooth vs. fan/corner closures — by adding pseudo-arclength continuation and the three diagnostics of §7 as `mdo/continuation.py`. It barely touches the MDO spine and is mostly sandbox-runnable NumPy, so it de-risks independently.

12.5 First increment (the walking skeleton)

1. `mdo/schema.py` + `mdo/scaling.py`: the variable/bounds/mission pytrees and a jited dummy `evaluate(x)` returning a scalar, so `jax.jacfgd` runs clean (Phase-1 gate).
2. `mdo/properties.py`: tabulate `cea_propellant` over $(P_c, O/F)$, fit splines, validate gradients against held-out CEA points (Phase-2 gate).
3. **Skeleton assembly.py**: wire the analytic nozzle (`primitives.py`) + `throat_wall_temperature` + algebraic injector + meanline pump into one R , solve with `least_squares_solve`, wrap in `make_differentiable_solution`, and take *one* end-to-end gradient. That gradient existing is the real unlock; everything after deepens a block behind a stable interface.

Reality checks. JAX BVP solves are *host-only* ($\sim 5\text{--}15$ min); the implicit-contour path and full coupled solves run on your machine, not in-session, while the analytic-contour skeleton is fast and testable anywhere. Keep the NumPy oracle for every block so CI gates parity without the slow solves. The honest long pole remains **J3b** (the differentiable kernel/BDE march), but it gates *only* wall-coordinate sensitivities on the implicit path — the skeleton and the Paper-1 Pareto do not wait on it.

13 Open risks and decision log

1. **Contour fidelity (decided).** TOP-Bézier for bulk Pareto; implicit variational only for existence-boundary and exact-vs-TOP studies. J3b (differentiable kernel/BDE march) gates *only* wall-coordinate sensitivities on the implicit path.
2. **Thermal fidelity ladder (decided).** 4a before 4b; do not gate first coupled solves on quasi-2-D.

3. **c^* convention (decided).** $c_{\text{delivered}}^* = \eta_{c^*} c_{\text{ideal}}^*$, pinned in `properties.py`; no double counting in the injector closure.
4. **Coupling attribution (open).** No unique additive decomposition; define an ablation protocol (freeze one feedback edge, re-optimize, measure Δ) and report order-dependence.
5. **Multipoint validity (open).** Evaluate separation, property, and efficiency surfaces at *every* operating point; deep throttle stresses the separation screen and the off-BEP pump surface simultaneously.
6. **Coupled-state conditioning (open).** Feedback cycles make the monolithic Newton solve stiff; budget homotopy/continuation for the state solve itself, independent of the cliff study.
7. **SP-8089 opacity (noted).** Image scan; OCR required before any correlation extraction.

References

- [1] G. V. R. Rao, “Exhaust Nozzle Contour for Optimum Thrust,” *Jet Propulsion* 28(6), 377–382, 1958. `propulsion_texts/rao1958.pdf`.
- [2] G. V. R. Rao, K. Beck, J. Booth, “Nozzle Optimization for Space-Based Vehicles,” 1999. `propulsion_texts/rao1999.pdf`.
- [3] G. V. R. Rao, “Recent Developments in Rocket Nozzle Configurations,” *ARS J.*, 1961. `propulsion_texts/RaoRecentDevinRockNozConfig.pdf`.
- [4] D. K. Huzel, D. H. Huang, *Design of Liquid Propellant Rocket Engines*, NASA SP-125. `propulsion_texts/19710019929.pdf`.
- [5] *Liquid Rocket Engine Nozzles*, NASA SP-8120. `propulsion_texts/19770009165.pdf`.
- [6] *Liquid Rocket Engine Fluid-Cooled Combustion Chambers*, NASA SP-8087. `propulsion_texts/19730022965.pdf`.
- [7] *Liquid Rocket Engine Injectors*, NASA SP-8089. `propulsion_texts/pintle_injector/19760023196.pdf` (image scan).
- [8] *Liquid Rocket Engine Centrifugal Flow Turbopumps*, NASA SP-8109. `propulsion_texts/fuel_pump_design/19740020848.pdf`.
- [9] *Liquid Rocket Engine Turbopump Inducers*, NASA SP-8052. `propulsion_texts/fuel_pump_design/19710025474.pdf`.
- [10] *Liquid Propellant Rocket Combustion Instability*, NASA SP-194. `propulsion_texts/19720026079.pdf`.
- [11] D. R. Bartz, “A Simple Equation for Rapid Estimation of Rocket Nozzle Convective Heat Transfer Coefficients,” *Jet Propulsion*, 1957. `propulsion_texts/technical-notes-1957.pdf`.
- [12] J. D. Anderson, *Modern Compressible Flow*, McGraw-Hill. `propulsion_texts/5f36b7c4\protect.\kern\fontdimen3\font.\kern\fontdimen3\font.\kern\fontdimen3\font.pdf`.
- [13] F. M. White, *Fluid Mechanics*, 7th ed., McGraw-Hill.
- [14] M. Pizzarelli, S. Carapellese, F. Nasuti, “A Quasi-2-D Model for the Prediction of the Wall Temperature of Rocket Engine Cooling Channels,” *Numerical Heat Transfer A*, 2011. `propulsion_texts/pizzarelli2011.pdf`.
- [15] J. Carlile, R. Quentmeyer, “An Experimental Investigation of High-Aspect-Ratio Cooling Passages,” AIAA, 1992. `propulsion_texts/carlile1992.pdf`.

- [16] M. Son et al., “Design Procedure of a Movable Pintle Injector for Liquid Rocket Engines,” *J. Propulsion and Power*, 2017. [propulsion_texts/pintle_injector/son2017.pdf](#).
- [17] M. Son et al., “Effects of Momentum Ratio and Weber Number on Spray Half Angles of a Pintle Injector,” *J. Thermal Science*, 2015. [propulsion_texts/pintle_injector/s11630-015-0753-7.pdf](#).
- [18] J. Lee et al., “Design and mass analysis of an electric-pump feed system,” *Int. J. Aeronautical and Space Sciences*, 2021. [propulsion_texts/fuel_pump_design/s42405-020-00325-z.pdf](#).
- [19] *Low-Cycle Fatigue of NARloy-Z*, NASA CR-134627. [propulsion_texts/materials_science/19740017910.pdf](#).
- [20] J.S. Porowski et al., “Simplified Design Methods for Low-Cycle Fatigue of Thrust Chambers,” 1985. [propulsion_texts/porowski1985.pdf](#).
- [21] S. Gordon, B.J. McBride, *Computer Program for Calculation of Complex Chemical Equilibrium Compositions and Applications*, NASA RP-1311, 1994.
- [22] J.R.R.A. Martins, A.B. Lambe, “Multidisciplinary Design Optimization: A Survey of Architectures,” *AIAA J.* 51(9), 2013.
- [23] J.R.R.A. Martins, J.T. Hwang, “Review and Unification of Methods for Computing Derivatives of Multidisciplinary Computational Models,” *AIAA J.* 51(11), 2013.
- [24] J.T. Hwang, J.R.R.A. Martins, “A Computational Architecture for Coupling Heterogeneous Numerical Models and Computing Coupled Derivatives,” *ACM TOMS* 44(4), 2018. <https://dl.acm.org/doi/10.1145/3182393>.
- [25] J.S. Gray, J.T. Hwang, J.R.R.A. Martins, et al., “OpenMDAO: an open-source framework for multidisciplinary design, analysis, and optimization,” *Struct. Multidiscip. Optim.* 59, 2019. <https://link.springer.com/article/10.1007/s00158-019-02211-z>.
- [26] “Benchmarking of monolithic MDO formulations and derivative computation techniques using OpenMDAO,” *Struct. Multidiscip. Optim.*, 2020. <https://link.springer.com/article/10.1007/s00158-020-02521-7>.
- [27] J.R.R.A. Martins, A. Ning, *Engineering Design Optimization*, Cambridge Univ. Press, 2021.
- [28] M. Blondel et al., “Efficient and Modular Implicit Differentiation,” NeurIPS 2022; arXiv:2105.15183. <https://arxiv.org/pdf/2105.15183>.
- [29] J. Rader, T. Lyons, P. Kidger, “Optimistix: modular optimisation in JAX and Equinox,” arXiv:2402.09983, 2024. <https://arxiv.org/pdf/2402.09983>.
- [30] H.B. Keller, “Numerical solution of bifurcation and nonlinear eigenvalue problems,” in *Applications of Bifurcation Theory*, Academic Press, 1977.
- [31] A. Wächter, L.T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming (IPOPT),” *Math. Program.* 106, 2006.
- [32] A. Griewank, A. Walther, *Evaluating Derivatives*, 2nd ed., SIAM, 2008.